SymPy Bug Report

1 Bug 4: Integral of $\log(x)/(x^2-1)$ Differentiates Back with an Extra Imaginary Term

1.1 Minimal Reproducer

```
import os
import sys

SYMPY_CHECKOUT_PATH = os.environ.get("SYMPY_CHECKOUT_PATH")
if SYMPY_CHECKOUT_PATH:
    sys.path.insert(0, SYMPY_CHECKOUT_PATH)

import sympy
from sympy import *  # noqa: F401,F403

print("SymPy version:", sympy.__version__)
print("SymPy file:", sympy.__file__)
print("Python executable:", sys.executable)
if SYMPY_CHECKOUT_PATH and not os.path.abspath(sympy.__file__).startswith(os.path.abspath
    (SYMPY_CHECKOUT_PATH)):
    raise RuntimeError("SymPy was not imported from SYMPY_CHECKOUT_PATH")

x = symbols("x")
f = log(x)/(x**2 - 1)
F = integrate(f, x)
residual = simplify(diff(F, x) - f)
print("integrate(log(x)/(x**2 - 1), x) =", F)
print("diff(integral, x) - log(x)/(x**2 - 1) =", residual)
print("residual at x=2 =", N(residual.subs(x, 2), 50))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
SymPy file: $SYMPY_CHECKOUT_PATH/sympy/__init__.py
Python executable: python3
residual at x=2 = 0.e-56 - 1.5707963267948966192313216916397514420985846996876*I
```

The returned antiderivative differentiates to the original real integrand plus a nonzero imaginary residual at the ordinary positive real point $x = 2$. The residual is $-1.57079632679\ldots i = -\pi i/2$, so this is not a formatting difference, an unevaluated zero, or a harmless alternative constant of integration.

1.3 Expected Behavior

For

$$f(x) = \frac{\log x}{x^2 - 1},$$

any valid local antiderivative F on a neighborhood of $x = 2$ should satisfy

$$F'(2) = f(2) = \frac{\log 2}{3}.$$

Equivalently, $(F' - f)|_{x=2}$ should simplify or numerically evaluate to 0.

1.4 Mathematical Explanation

The integrand

$$f(x) = \frac{\log x}{x^2 - 1}$$

is real-valued and analytic near $x = 2$, since $x = 2$ is away from the poles at $x = \pm 1$ and away from the principal logarithm branch point. The defining local property of an indefinite integral is therefore unambiguous: on such a neighborhood, $F'(x) = f(x)$.

SymPy's returned expression instead gives

$$F'(2) - f(2) = -\frac{\pi i}{2}.$$

An additive constant in an antiderivative would disappear under differentiation, so the residual cannot be explained by a different integration constant. It is also not merely a different branch-equivalent representation at the tested point: the derivative itself contains an extra branch term. Since $f(2) = \log(2)/3$ is real, the nonzero imaginary residual makes the returned expression fail the direct antiderivative check.

1.5 Source-Level Diagnosis

The diagnosis locates the relevant code in `sympy/integrals/meijerint.py`, specifically the shifted-logarithm lookup rule around line 196 and the Meijer indefinite integration functions `meijerint_indefinite` and `_meijerint_indefinite_1` around lines 1653–1740. The public call starts with the original integration request and enters `Integral._eval_integral`. Direct Risch and heurisch paths do not solve the original integrand. Then `manualintegrate` applies integration by parts, reducing the problem to logarithmic subintegrals:

```
integrate(log(x)/(x**2 - 1), x)

(log(x - 1)/2 - log(x + 1)/2)*log(x)
  - Integral(log(x - 1)/x, x)/2
  + Integral(log(x + 1)/x, x)/2
```

Because this expression still contains unevaluated `Integral` terms, `Integral._eval_integral` recursively evaluates those subintegrals. The diagnosis identifies `Integral(log(x - 1)/x, x)` as the faulty subproblem: it is solved by the Meijer integration path after rewriting $\log(x-1)$ through the table entry for shifted logarithms. For this shifted logarithm, `_rewrite_single` returns a branch-sensitive representation involving $x \exp_{\text{polar}}(-i\pi)$ and explicit $i\pi$ constants.

The resulting sub-antiderivative differentiates on the positive side of the branch point to

$$\frac{\log(x-1)}{x} + \frac{i\pi}{x},$$

rather than to $\log(x-1)/x$. In the full integration-by-parts result, this subintegral appears with coefficient $-1/2$, so the extra derivative term contributes an incorrect branch residual. At $x = 2$, the observed residual is $-\pi i/2$. The diagnosis therefore points to a branch-sensitive Meijer/polylogarithm expression being accepted as an antiderivative for the principal-log integrand on a real interval where it differentiates to the wrong branch.

A plausible fix direction supported by the diagnosis is for the Meijer rewrite for shifted logarithms either to preserve the principal branch of $\log(x-1)$ under the shift/sign rewrite or to decline this transformation when the shift crosses the logarithm branch cut. A conservative implementation could leave `meijerint_indefinite(log(x - 1)/x, x)` unevaluated unless it can attach correct branch conditions.

1.6 Additional Failing Instances

The independent verification covers the family

$$f_a(x) = \frac{\log x}{x^2 - a^2}, \qquad a \in \{1, 2, 3, 4, 5, 6\},$$

with the derivative residual checked at the regular positive real point $x = 2a$. The expected behavior is the same in every case: f_a is finite and analytic at $x = 2a$, and a valid local antiderivative must differentiate back to f_a, giving residual 0.

Input / Expression	**SymPy behavior**	**Expected behavior**
$\int \log(x)/(x^2-1)\,dx$, checked at $x=2$	residual $= -\pi i/2$	residual $= 0$
$\int \log(x)/(x^2-4)\,dx$, checked at $x=4$	residual $= -\pi i/4$	residual $= 0$
$\int \log(x)/(x^2-9)\,dx$, checked at $x=6$	residual $= -\pi i/9$	residual $= 0$
$\int \log(x)/(x^2-16)\,dx$, checked at $x=8$	residual $= -\pi i/16$	residual $= 0$
$\int \log(x)/(x^2-25)\,dx$, checked at $x=10$	residual $= -\pi i/25$	residual $= 0$
$\int \log(x)/(x^2-36)\,dx$, checked at $x=12$	residual $= -\pi i/36$	residual $= 0$

These cases show that the failure is not isolated to a single denominator or evaluation point. Across this quadratic family, the integration procedure introduces a branch contribution from shifted logarithmic subintegrals, while the mathematical antiderivative contract requires the residual to vanish at each tested regular real point.

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. It found broad public material on logarithmic branch cuts, polylogarithmic expressions, Meijer-G integration, and SymPy's indefinite integration contract. A focused re-check identified the open issue sympy/sympy#13850 ("Integration result appears to use a different branch of polylog"), which reports the same shifted-log/polylog branch-cut antiderivative family for $\int \log(x)/(1+x)\,dx$; the present bug reduces by parts to exactly the $\int \log(x \pm 1)/x\,dx$ subintegrals, so it is the same known family (see also

sympy/sympy#23337 for a related spurious $i\pi$ integration artifact). No prior report contained the exact expression $\log(x)/(x^2-1)$, the check at $x=2$, the residual $-\pi i/2$, or the $\log(x)/(x^2-a^2)$ family, so this specific instance is newly reported. The deduplication verdict is therefore a known family with a new specific instance, and this exact reproducible case remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import N, diff, integrate, log, simplify, symbols

@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_log_over_quadratic_antiderivative_differentiates_back(a):
    x = symbols("x")
    f = log(x)/(x**2 - a**2)
    F = integrate(f, x)
    residual = simplify(diff(F, x) - f)
    assert abs(complex(N(residual.subs(x, 2*a), 40))) < 1e-30
```